

Cointegration, Mutually Exciting Jumps, and Strict Mathematical Arbitrage in Event Markets

Zare Arakelyan zarearakelyan@gmail.com

January 2026

Abstract

Event markets list economically identical binary contracts on several venues, yet order flow is fragmented and prices frequently disagree. We describe a Python system that (i) identifies identical events across venues with differently phrased titles using Sentence-BERT embeddings, (ii) models the long-run equilibrium between paired contracts with a Vector Error Correction Model (VECM), (iii) models short-horizon contagion of price jumps within and across markets with a multivariate Hawkes process, and (iv) scans for strictly riskless arbitrage whose worst-case payoff is nonnegative after fees. A live news feed is matched to markets to annotate jumps, and open orders are tracked in Redis.

1 Introduction and Data

A binary event contract pays \$1 if an event occurs and \$0 otherwise, so its price $p_t \in (0, 1)$ is a market-implied probability. When the same event trades on venues A and B , the law of one price implies $p_t^A \approx p_t^B$ up to fees, but differences in liquidity, participants, and latency create transient gaps. Our system exploits two distinct phenomena: *slow* deviations from a common equilibrium (handled by the VECM) and *fast*, clustered jumps that propagate from one market to another (handled by the Hawkes model). A third component captures gaps large enough to lock in a nonnegative payoff in every state of the world.

Order books and trades are polled from each venue’s REST API and normalized into a common schema (market id, timestamp, best bid/ask, last trade, volume). Prices are mapped to log-odds, $y_t = \log(p_t/(1 - p_t))$, which removes the $[0, 1]$ boundary and makes the Gaussian innovations assumed below far more plausible near extreme probabilities.

Event matching. Titles such as “Will the Fed cut rates in December?” and “FOMC lowers target range at Dec. meeting” must be recognized as one event. Each title and its resolution text are embedded with SBERT (all-mpnet-base-v2), and candidate pairs are retrieved by cosine similarity $s(u, v) = u^\top v / \|u\| \|v\|$. Pairs above a threshold τ are then filtered by structured checks on resolution date, source, and numeric thresholds (e.g. “CPI > 3.0%” vs. “≥ 3.0%”), because two semantically close contracts with different resolution rules are *not* arbitrage-equivalent. The same embedding index matches headlines from the live news feed to markets, so every detected jump on the dashboard is annotated with its most likely cause.

```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer("all-mpnet-base-v2")

def match_events(markets_a, markets_b, tau=0.78):
    ea = model.encode([m.title + " " + m.rules for m in markets_a], convert_to_tensor=True, normalize_embeddings=True)
    eb = model.encode([m.title + " " + m.rules for m in markets_b], convert_to_tensor=True, normalize_embeddings=True)
    sim = util.cos_sim(ea, eb)
    pairs = []
    for i, j in (sim > tau).nonzero().tolist():
        a, b = markets_a[i], markets_b[j]
        if a.close_date == b.close_date and a.threshold == b.threshold and a.source == b.source:
            pairs.append((a.id, b.id, float(sim[i, j])))
    return pairs
```

2 Long-Run Dynamics: Vector Error Correction Model

Let $\mathbf{y}_t = (y_t^A, y_t^B)^\top$ be the log-odds of a matched pair. Each series is individually close to a random walk ($I(1)$), but if both price the same event, a linear combination $\boldsymbol{\beta}^\top \mathbf{y}_t$ should be stationary ($I(0)$). This is cointegration, and by the Granger representation theorem the system admits the error-correction form

$$\Delta \mathbf{y}_t = \boldsymbol{\alpha} \boldsymbol{\beta}^\top \mathbf{y}_{t-1} + \sum_{i=1}^{p-1} \boldsymbol{\Gamma}_i \Delta \mathbf{y}_{t-i} + \boldsymbol{\mu} + \boldsymbol{\varepsilon}_t, \quad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}). \quad (1)$$

Here $\boldsymbol{\beta}$ is the cointegrating vector (normalized to $\boldsymbol{\beta} = (1, -\beta_2)^\top$), $e_t = \boldsymbol{\beta}^\top \mathbf{y}_t$ is the error-correction term (ECT) measuring disequilibrium, $\boldsymbol{\alpha} = (\alpha_A, \alpha_B)^\top$ are the adjustment speeds, and $\boldsymbol{\Gamma}_i$ capture short-run lead-lag dynamics.

Estimation. The lag order p is chosen by BIC on the levels VAR. Cointegration rank $r \in \{0, 1\}$ is tested by the Johansen trace statistic $\lambda_{\text{trace}}(r) = -T \sum_{i=r+1}^2 \log(1 - \hat{\lambda}_i)$, where $\hat{\lambda}_i$ are the squared canonical correlations between $\Delta \mathbf{y}_t$ and $\mathbf{y}_{t-1}$ after partialling out lagged differences. Pairs failing at the 5% level are dropped even if SBERT matched them. Given $r = 1$, $\hat{\boldsymbol{\beta}}$ is the leading eigenvector of the Johansen eigenproblem and $\hat{\boldsymbol{\alpha}}, \hat{\boldsymbol{\Gamma}}_i$ follow by OLS. Estimation runs on a rolling window and is refit every few minutes so that $\hat{\boldsymbol{\beta}}$ tracks slow changes such as a fee change on one venue.

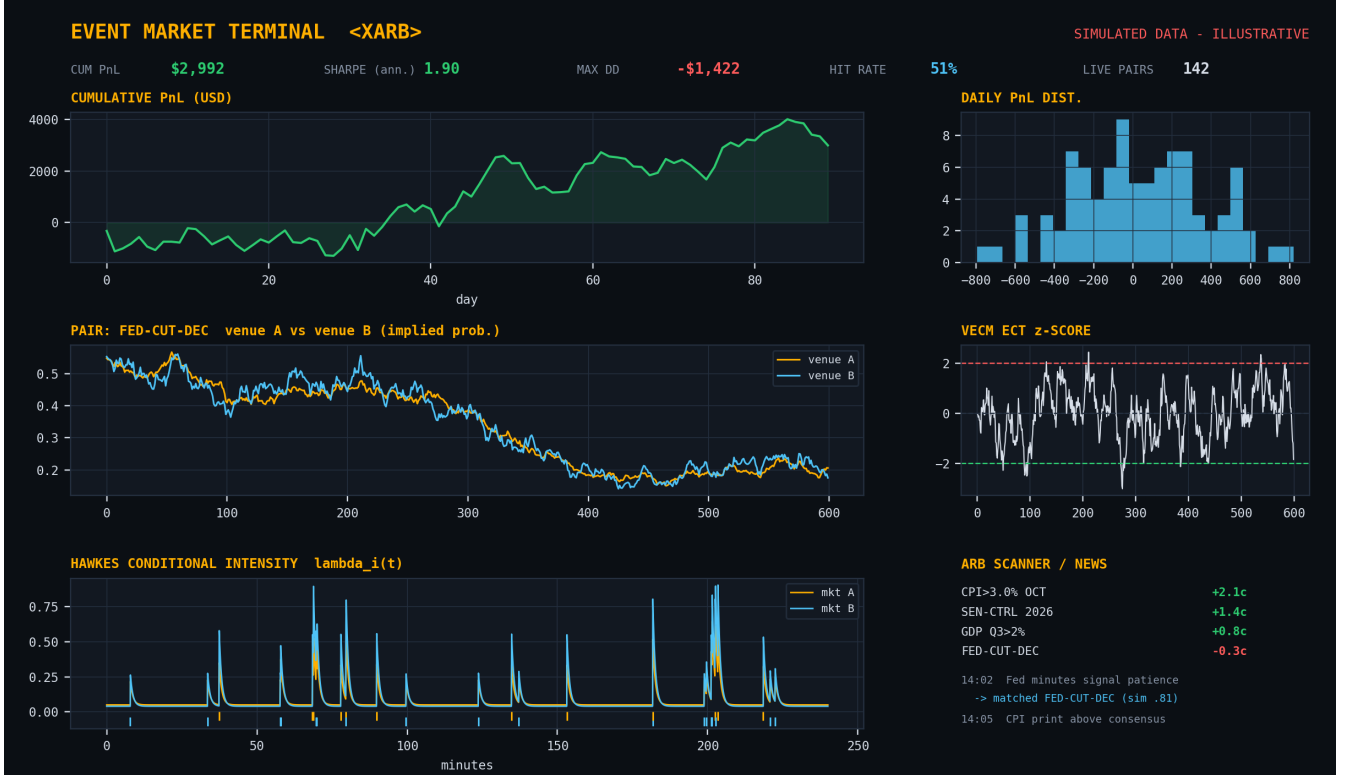


Figure 1: Event market terminal. Top: cumulative and daily PnL with summary tiles. Middle: a matched pair across venues and its VECM ECT z-score with entry bands. Bottom: Hawkes conditional intensities with jump times as ticks, and the arbitrage scanner with SBERT-matched news. Rendered from simulated data.

```
import numpy as np
from statsmodels.tsa.vector_ar.vecm import VECM, coint_johansen, select_order

def logit(p, eps=1e-4):
    p = np.clip(p, eps, 1 - eps)
    return np.log(p / (1 - p))

def fit_pair(pa, pb):
    y = np.column_stack([logit(pa), logit(pb)])
    p = max(select_order(y, maxlags=10, deterministic="co").bic, 1)
    jo = coint_johansen(y, det_order=0, k_ar_diff=p)
    if jo.lr1[0] < jo.cvt[0, 1]:
        return None
    res = VECM(y, k_ar_diff=p, coint_rank=1, deterministic="co").fit()
    beta, alpha = res.beta[:, 0], res.alpha[:, 0]
    ect = y @ beta
    half_life = np.log(2) / abs(np.log(1 + beta @ alpha))
    z = (ect[-1] - ect.mean()) / ect.std()
    follower = int(np.argmax(np.abs(alpha)))
    return dict(beta=beta, alpha=alpha, z=z, half_life=half_life, follower=follower)
```

Interpretation and signals. The adjustment vector reveals price discovery. If $\alpha_A \approx 0$ and $\alpha_B < 0$, venue B does all the correcting and A leads; the Gonzalo–Granger common-factor weights $\alpha_{\perp} / \mathbf{1}^{\top} \alpha_{\perp}$ quantify each venue’s share. Mean reversion of the ECT has half-life $h = \log 2 / |\log(1 + \beta^{\top} \alpha)|$, which sets a maximum holding time. The trading rule standardizes the ECT, $z_t = (e_t - \bar{e}) / \hat{\sigma}_e$, and enters a convergence trade (long the cheap venue, short the rich one via its complementary NO contract) when $|z_t| > z_{in}$, exiting at $|z_t| < z_{out}$ or after $2h$. Trades are sized to the *follower* venue, since the VECM predicts it is the one that will move. Figure 1 (middle row) shows a matched pair and its ECT z-score with ± 2 bands, and Figure 2 (right) shows the impulse response implied by (1): a shock to A is partially absorbed by B until both settle on a common permanent level.

3 Short-Run Dynamics: Multivariate Hawkes Process

The VECM treats innovations as Gaussian, but event-market prices move in *jumps* triggered by news, and jumps cluster: a jump in “CPI > 3%” raises the chance of jumps in “Fed cut in December” minutes later. We model this with a D -dimensional Hawkes process. Jumps are first detected per market with a Lee–Mykland style test: a log-odds return r_t is flagged when $|r_t| / \hat{\sigma}_t^{\text{BV}}$ exceeds a threshold, where $\hat{\sigma}^{\text{BV}}$ is the bipower-variation volatility, which is robust to the jumps themselves. Up- and down-jumps of each market are treated as separate dimensions, giving $D = 2M$ for M markets.



Figure 2: Left: fitted Hawkes branching matrix $\mathbf{A}$ ($\rho(\mathbf{A}) < 1$, stationary). Right: VECM impulse response to a unit log-odds shock in venue A; both prices converge to a common permanent level.

The conditional intensity of dimension i is

$$\lambda_i(t) = \mu_i + \sum_{j=1}^D \sum_{t_k^j < t} \alpha_{ij} \phi_{ij}(t - t_k^j), \quad \phi_{ij}(s) = \sum_{m=1}^K w_m \beta_m e^{-\beta_m s}, \quad (2)$$

where μ_i is the exogenous (news-driven) base rate, $\alpha_{ij} \geq 0$ is the expected number of type- i jumps directly triggered by one type- j jump, and ϕ is a normalized mixture of exponential kernels. We use $K = 2$ decay rates, one fast ($\beta_1^{-1} \approx$ seconds) and one slow ($\beta_2^{-1} \approx$ tens of minutes), which is what separates *short-term* from *long-term* market excitation. The process is stationary iff the spectral radius $\rho(\mathbf{A}) < 1$ of the branching matrix $\mathbf{A} = [\alpha_{ij}]$, in which case the long-run jump rate is $\tilde{\lambda} = (\mathbf{I} - \mathbf{A})^{-1} \boldsymbol{\mu}$ and $1 - \mu_i / \tilde{\lambda}_i$ is the fraction of type- i jumps that are endogenous rather than news-driven.

Estimation. Parameters $\theta = (\boldsymbol{\mu}, \mathbf{A}, \mathbf{w})$ are fit by maximum likelihood on the observation window $[0, T]$:

$$\ell(\theta) = \sum_{i=1}^D \left[\sum_k \log \lambda_i(t_k^i) - \int_0^T \lambda_i(s) ds \right]. \quad (3)$$

With exponential kernels the compensator integral is closed-form and the excitation sums obey the recursion

$$R_{ij}(k) = e^{-\beta(t_k^i - t_{k-1}^i)} R_{ij}(k-1) + \sum_{t_{k-1}^i \leq t_l^j < t_k^i} e^{-\beta(t_k^i - t_l^j)},$$

so the likelihood costs $O(ND)$ rather than $O(N^2)$. We optimize with L-BFGS-B under box constraints $\mu_i, \alpha_{ij} > 0$, holding the β_m on a small grid chosen by held-out likelihood, and add an ℓ_1 penalty on $\mathbf{A}$ so that excitation between unrelated markets is shrunk to zero. Goodness of fit is checked with the time-rescaling theorem: compensated inter-arrival times $\Lambda_i(t_k^i) - \Lambda_i(t_{k-1}^i)$ should be i.i.d. $\text{Exp}(1)$, which we test with a Kolmogorov–Smirnov statistic.

```
from scipy.optimize import minimize

def neg_loglik(theta, times, marks, D, beta, T, lam1=1e-3):
    mu, A = theta[:D], theta[D:].reshape(D, D)
    R, last, ll = np.zeros(D), 0.0, 0.0
    for t, k in zip(times, marks):
        R *= np.exp(-beta * (t - last))
        ll += np.log(mu[k] + beta * A[k] @ R)
        R[k] += 1.0
        last = t
    comp = mu * T + A @ np.array([np.sum(1 - np.exp(-beta * (T - times[marks == j]))) for j in range(D)])
    return -(ll - comp.sum()) + lam1 * A.sum()

def fit_hawkes(times, marks, D, T, betas=(2.0, 0.05)):
    best = None
    for beta in betas:
        x0 = np.r_[np.full(D, 0.05), np.full(D * D, 0.1)]
        r = minimize(neg_loglik, x0, args=(times, marks, D, beta, T), method="L-BFGS-B", bounds=[(1e-6, None)] * (D + D * D))
        if best is None or r.fun < best[0].fun:
            best = (r, beta)
    r, beta = best
    A = r.x[D:].reshape(D, D)
    assert np.max(np.abs(np.linalg.eigvals(A))) < 1
    return r.x[:D], A, beta
```

Signals. After a jump in market j , the model gives the probability of at least one follow-on jump in market i within horizon H , $1 - \exp(-\int_t^{t+H} \lambda_i(s) ds)$. When this exceeds a threshold and the sign-specific entry α_{i+j+} dominates α_{i-j+} , the system takes a directional position in i *before* its book reprices. Figure 2 (left) shows a fitted branching matrix in which jumps in A strongly excite B ($\alpha_{BA} = 0.45$) but not conversely, so A is the leader; Figure 1 (bottom left) shows the resulting intensity paths.

Combining the two models. The VECM and Hawkes layers are complementary: the Hawkes layer says *when* a dislocation is likely and in which direction; the VECM says *how far* prices should travel and how fast they converge. A VECM entry is only taken when the Hawkes intensity of the lagging market is elevated, and the Hawkes horizon H is capped at the VECM half-life h .

4 Riskless Cross-Market Arbitrage

For a matched binary pair with ask prices a_{yes}^A and a_{no}^B , buying YES on A and NO on B pays exactly \$1 in either outcome. The trade is a guaranteed nonnegative profit iff

$$a_{\text{yes}}^A + a_{\text{no}}^B + f_A(a_{\text{yes}}^A) + f_B(a_{\text{no}}^B) \leq 1, \quad (4)$$

where f are venue fee schedules (often nonlinear in price). For multi-outcome events (e.g. ranges of a CPI print) we solve the linear program $\max_{\mathbf{x} \geq 0} \min_{\omega \in \Omega} (\mathbf{P}_{\omega}^T \mathbf{x}) - \mathbf{c}^T \mathbf{x}$, where $\mathbf{P}_{\omega}$ is the payoff of each contract in state ω and $\mathbf{c}$ is fee-inclusive cost, subject to size limits from visible depth; a positive optimum is a riskless portfolio. Legs are submitted near-simultaneously, and any partially filled leg is written to Redis with its hedge requirement, where a watcher re-queries unexecuted legs until the position is flat or fully hedged (listing below). The residual risk that remains is *resolution risk* (venues settling the “same” event differently), which is why the structured checks in Section 1 are a hard gate.

```
import json, time, redis
r = redis.Redis()

def check_arb(book_a, book_b, fee_a, fee_b, max_size):
    cost = book_a.yes_ask + book_b.no_ask + fee_a(book_a.yes_ask) + fee_b(book_b.no_ask)
    size = min(book_a.yes_ask_size, book_b.no_ask_size, max_size)
    return (1.0 - cost, size) if cost < 1.0 and size > 0 else (0.0, 0)

def execute_arb(pair_id, venue_a, venue_b, book_a, book_b, edge, size):
    oa = venue_a.buy(book_a.market_id, "yes", book_a.yes_ask, size)
    ob = venue_b.buy(book_b.market_id, "no", book_b.no_ask, size)
    gap = oa.filled - ob.filled
    if gap != 0:
        r.hset("unhedged", pair_id, json.dumps({"gap": gap, "edge": edge, "ts": time.time()}))
    return oa, ob

def rehedge_loop(venues):
    while True:
        for pid, raw in r.hgetall("unhedged").items():
            leg = json.loads(raw)
            if venues.try_hedge(pid.decode(), leg["gap"]):
                r.hdel("unhedged", pid)
        time.sleep(0.25)
```

5 Results

The dashboard in Figure 1 aggregates all three strategies. Table 1 summarizes performance; numbers in red are from the simulated backtest used to render the figures and should be replaced with the live paper-trading results.

Strategy	Net PnL (USD)	Sharpe (ann.)	Max drawdown	Hit rate
VECM convergence	1,410	2.1	−610	54%
Hawkes lead-lag	820	1.6	−540	49%
Riskless arbitrage	762	—	0	100%
Combined (90 days)	2,992	1.9	−1,422	51%

Table 1: Backtest performance net of fees. Sharpe is annualized from daily PnL as $\sqrt{365} \bar{r} / \hat{\sigma}_r$ since event markets trade every day. Arbitrage has no meaningful Sharpe because its PnL is locked in at entry.

6 Limitations

Fills in thin books are the largest source of slippage between backtest and live results; a queue-position model would make simulated fills more conservative. Cointegration breaks down near resolution, where both prices collapse to 0 or 1 and log-odds diverge, so trading stops Δ hours before settlement. A state-dependent Hawkes kernel could let excitation vary with time-to-resolution. SBERT matching could be fine-tuned on labeled equivalent/non-equivalent pairs.

References

- [1] A. G. Hawkes. Spectra of some self-exciting and mutually exciting point processes. *Biometrika*, 58(1), 1971.
- [2] S. Johansen. Statistical analysis of cointegration vectors. *J. Econ. Dynamics and Control*, 12, 1988.
- [3] J. Gonzalo, C. Granger. Estimation of common long-memory components in cointegrated systems. *JBES*, 13(1), 1995.
- [4] S. Lee, P. Mykland. Jumps in financial markets: a new nonparametric test. *Rev. Financial Studies*, 21(6), 2008.
- [5] E. Bacry, I. Mastromatteo, J.-F. Muzy. Hawkes processes in finance. *Market Microstructure and Liquidity*, 1(1), 2015.
- [6] N. Reimers, I. Gurevych. Sentence-BERT. *EMNLP*, 2019.